

Design Pattern in Java

Most asked Interview Questions 2-3 year experience

[All patterns](#)[Creational](#)[Structural](#)[Behavioral](#)

14 patterns

[Creational](#) **Singleton**

One instance, global access point — classic but tricky in multithreaded environments.

[Show code + questions ↓](#)[Creational](#) **Factory Method**

Delegate object creation to subclasses — the creator decides the product.

[Show code + questions ↓](#)[Creational](#) **Abstract Factory**

Family of related objects without specifying their concrete classes.

[Show code + questions ↓](#)[Creational](#) **Builder**

Construct complex objects step-by-step. Separates construction from representation.

[Show code + questions ↓](#)[Creational](#) **Prototype**

Clone existing objects instead of creating new ones from scratch.

[Show code + questions ↓](#)[Structural](#) **Adapter**

Makes incompatible interfaces work together — the 'wrapper' pattern.

[Show code + questions ↓](#)[Structural](#) **Decorator**

Add behaviour to objects dynamically without subclassing.

[Show code + questions ↓](#)[Structural](#) **Facade**

Simplified interface to a complex subsystem — hides complexity behind one class.

[Show code + questions ↓](#)[Structural](#) **Proxy**

A surrogate that controls access to the real object — lazy loading, security, logging.

[Show code + questions ↓](#)[Behavioral](#) **Observer**

Notify multiple objects when one object changes state — event-driven systems.

[Show code + questions ↓](#)[Behavioral](#) **Strategy**

Define a family of algorithms, encapsulate each one, make them interchangeable.

[Show code + questions ↓](#)[Behavioral](#) **Template Method**

Define the skeleton of an algorithm in a base class; subclasses fill in the steps.

[Show code + questions ↓](#)[Behavioral](#) **Command**

Encapsulate a request as an object — supports undo, queuing, and logging.

[Show code + questions ↓](#)[Behavioral](#) **Chain of Responsibility**

Pass requests along a chain of handlers until one handles it.

[Show code + questions ↓](#)

Here's a full interactive guide covering the 14 most commonly asked design pattern questions for 2–3 years of Java experience. Click any card to expand code + interview questions, and use the tabs to filter by category.

How interviews typically go at your experience level:

Interviewers won't just ask "what is Singleton" — they probe *tradeoffs*. Expect questions like:

- *"You used Singleton in your project — how did you handle thread safety?"*
- *"Why did you choose Strategy over if-else here?"*
- *"Where have you actually seen Decorator in Java's standard library?"*

The patterns that come up most at 2–3 years:

Singleton (with thread safety nuance), Builder (especially Lombok), Observer (event systems), Strategy (Comparator, lambdas), and Proxy (Spring AOP) are almost guaranteed. Factory Method vs Abstract Factory is a very common "gotcha" question.

One tip: Always anchor your answer with a real JDK or Spring example —

`InputStreamReader` for Adapter, `JdbcTemplate` for Template Method, `Collections.sort` for Strategy. It signals genuine understanding rather than textbook memorization.

Explain Singleton pattern in Java with a real-world scenario and tricky interview questions ↗

REAL-WORLD SCENARIO

Database connection pool manager

Your app needs exactly one pool manager to hand out DB connections. Creating multiple pool managers wastes resources and causes connection leaks. Singleton ensures only one manager exists, shared across all threads.

1 — NAIVE (BROKEN IN MULTITHREADING) NOT THREAD-SAFE

```
public class DBPoolManager {
    private static DBPoolManager instance;

    private DBPoolManager() { /* init pool */ }

    public static DBPoolManager getInstance() {
        if (instance == null)           // Thread A and B both pass this check!
            instance = new DBPoolManager(); // Two instances created. Bug.
        return instance;
    }
}
```

2 — DOUBLE-CHECKED LOCKING + VOLATILE PREFERRED

```
public class DBPoolManager {
    private static volatile DBPoolManager instance; // volatile = JMM guarantee

    private DBPoolManager() {}

    public static DBPoolManager getInstance() {
        if (instance == null) {           // First check (no lock)
            synchronized (DBPoolManager.class) {
                if (instance == null)     // Second check (inside lock)
                    instance = new DBPoolManager();
            }
        }
        return instance;
    }
}
```

3 — BILL PUGH (LAZY + SAFE, NO SYNCHRONIZATION) ELEGANT

```
public class DBPoolManager {
    private DBPoolManager() {}

    private static class Holder {           // Loaded only when first accessed
        static final DBPoolManager INSTANCE
            = new DBPoolManager();         // JVM guarantees class-init is atomic
    }

    public static DBPoolManager getInstance() {
        return Holder.INSTANCE;
    }
}
```

```
}
```

4 — ENUM SINGLETON REFLECTION-PROOF + SERIALIZATION-SAFE

```
public enum DBPoolManager {  
    INSTANCE;  
  
    public void getConnection() { /* ... */ }  
}  
// Usage: DBPoolManager.INSTANCE.getConnection();  
// JVM guarantees one instance. Serialization handled automatically.
```

Here's a full deep-dive — use the four tabs to explore each angle:

Implementations — four variants from broken to bulletproof, with syntax highlighting so you can see exactly what each adds and why.

Thread race simulator — click "Run race (naive)" a few times and watch threads collide. Then "Run race (volatile + lock)" to see the safe version. The naive version won't always produce a race (that's what makes it so dangerous in production — it works fine in testing), but rerun a few times and you'll see multiple instances appear.

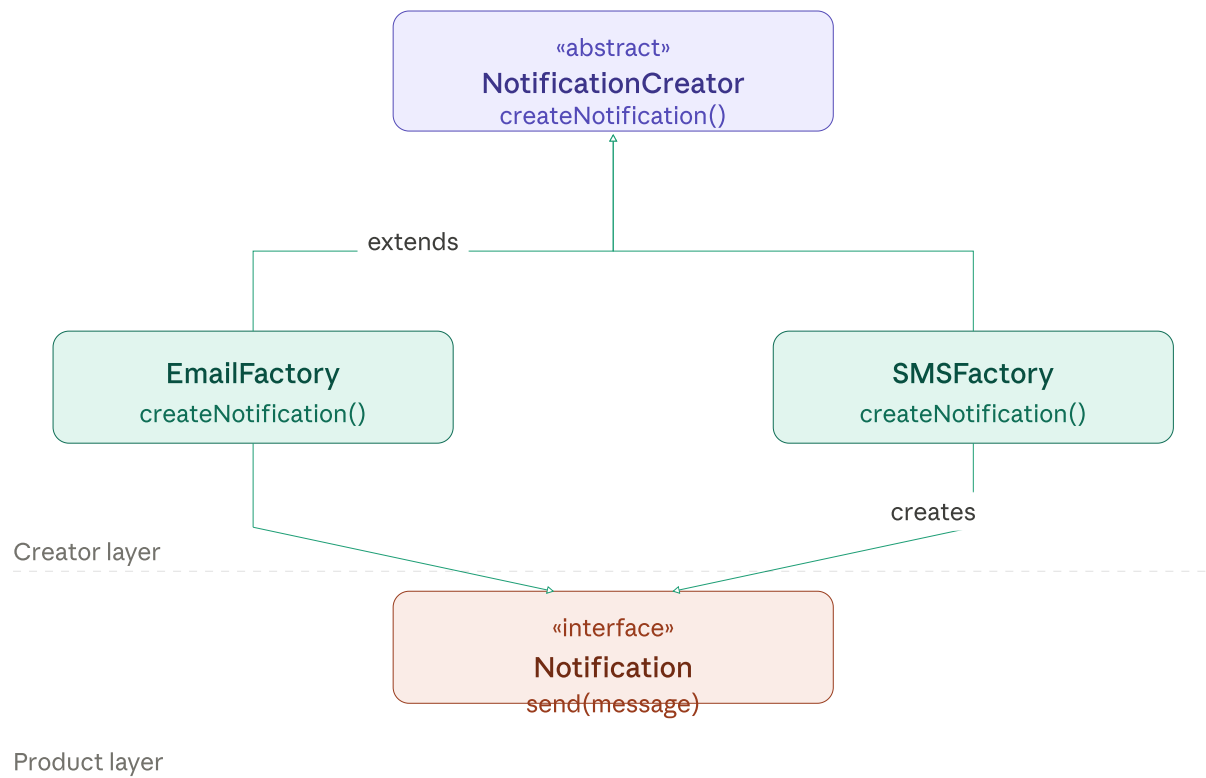
How to break it — the three classic gotchas interviewers love: reflection, serialization, and ClassLoaders. These are the "gotcha" questions that separate candidates who read a textbook from those who've actually thought about it.

Tricky questions — five of the most common follow-up questions, each with a hidden answer you can reveal. The volatility question and the Spring-vs-GoF question trip up a lot of 2–3 year engineers.

The one thing that impresses interviewers most at your level: knowing that Singleton is actually an *anti-pattern when overused* — global mutable state, hard to test, tight coupling. Knowing when *not* to use it, and proposing dependency injection as the alternative, signals maturity beyond "I can implement it."

Explain Factory Method pattern in Java with a real-world scenario and tricky interview questions ↗

Now let me build the full interactive deep-dive — first a structural diagram showing the class hierarchy, then the interactive widget.



The diagram above shows the two-layer structure at the heart of Factory Method. The **Creator layer** holds the abstract factory and its concrete subclasses. The **Product layer** holds the interface and its concrete implementations. Each `ConcreteCreator` knows exactly which `ConcreteProduct` to build — the client only ever talks to the abstractions on both sides.

Now here's the full interactive deep-dive with real-world code, a live playground, and tricky interview questions:

[Real-world scenario](#)[Full Java code](#)[Live playground](#)[vs Abstract Factory](#)[Tricky questions](#)

REAL-WORLD SCENARIO — NOTIFICATION SERVICE

The problem

You're building a notification system that sends alerts via Email, SMS, and Push. The sending logic differs for each channel, but the "decide what to send" logic is always the same. You want to add new channels (WhatsApp, Slack) without touching existing code.

HOW THE FLOW WORKS

- 1 **Client** calls `factory.notify("Payment received")` — it doesn't know or care which channel is being used.
- 2 **Creator (abstract)** calls `createNotification()` — this is the factory method, defined abstract in the base class.
- 3 **ConcreteCreator** (e.g. `EmailFactory`) overrides `createNotification()` and returns an `EmailNotification` object.
- 4 **Product's** `send()` method is called — the creator's base class logic works with the `Notification` interface, not the concrete type.
- 5 **Adding a new channel** (e.g. Slack) requires only a new `SlackFactory` + `SlackNotification`. Zero changes to existing classes. Open/Closed Principle satisfied.

JDK EXAMPLES OF FACTORY METHOD IN THE WILD

`Calendar.getInstance()`

Returns a locale-appropriate subclass of `Calendar`. You get a `GregorianCalendar`, `BuddhistCalendar`, etc. depending on the locale — without ever asking for one explicitly.

`NumberFormat.getInstance()`

Factory method returns the right formatter subclass for the locale. The calling code works against `NumberFormat` and never needs to know the concrete type.

`Collection.iterator()`

Each collection (`ArrayList`, `LinkedList`) overrides `iterator()` to return its own iterator implementation. Classic Factory Method.

`DocumentBuilderFactory`

In JAXP, `DocumentBuilderFactory.newInstance()` is a factory method that returns the right parser implementation based on the environment.

Here's what's packed into each tab:

Real-world scenario walks through a notification service — Email, SMS, Push — and shows exactly how the pattern flows step by step, plus four real JDK examples where Factory Method actually lives (`Calendar.getInstance()`, `Collection.iterator()`, etc.).

Full Java code shows the complete five-class implementation: the `Notification` interface, three concrete products, the abstract `NotificationCreator`, and three factories — all with highlighted syntax so the structure reads clearly.

Live playground lets you pick a channel and message and see the factory resolve in real-time. Hit "Send via all channels" to see all four factories fire from the same client call. Notice the Slack channel — it was added without touching any existing class. That's the Open/Closed Principle visually demonstrated.

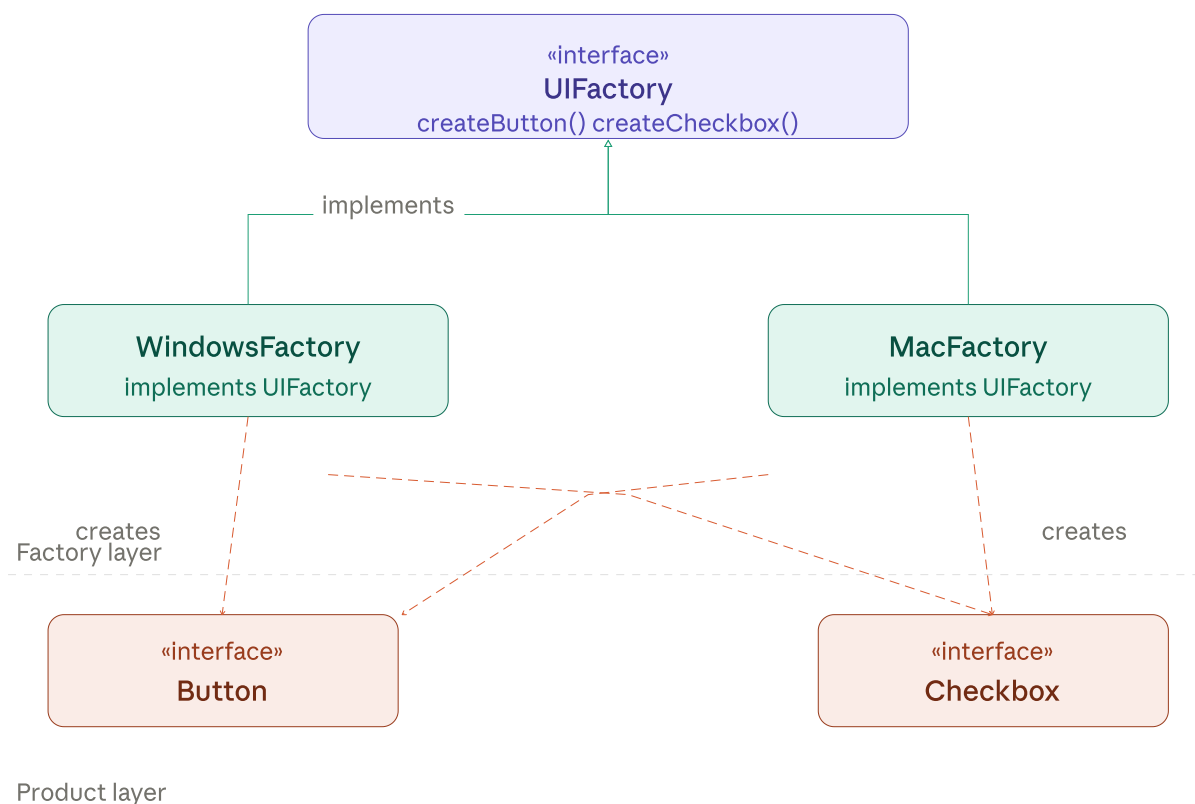
vs Abstract Factory covers the comparison interviewers love to ask. The one-line mental model: Factory Method = "let the subclass decide one product"; Abstract Factory = "give me a whole matching family."

Tricky questions has six of the hardest follow-ups, including the

`Calendar.getInstance()` trap (Q3) and the functional/lambda angle (Q5) that most 2-3 year engineers miss. The Spring `FactoryBean` answer in Q6 is the one that earns "senior" credibility in interviews.

Explain Abstract Factory pattern in Java with a real-world scenario and tricky interview questions ↗

First, a structural diagram showing the full class map — the defining feature of Abstract Factory is its 2×N grid of factories vs products:



The key thing to see in that diagram: Abstract Factory is a **grid**, not a tree. Columns = product families (Windows, Mac). Rows = product types (Button, Checkbox). Each concrete factory fills one full column — it guarantees every product it creates belongs to the same family. No mixing Mac buttons with Windows checkboxes.

Now the full interactive deep-dive:

[Real-world scenario](#)[Full Java code](#)[Live playground](#)[Pitfalls & extensions](#)[...](#)[Tricky questions](#)

REAL-WORLD SCENARIO — CROSS-PLATFORM UI TOOLKIT

The problem

You're building a desktop app that ships on Windows, macOS, and Linux. Each OS has its own native widgets — Windows uses rounded Metro buttons, Mac uses Aqua-style controls. If you mix platforms (Mac button + Windows checkbox on the same screen), the UI looks broken. You need a guarantee that every widget on a screen belongs to the same family.

Why not Factory Method?

Factory Method handles *one* product type. Here you need *multiple coordinated products* — Button, Checkbox, Dialog, ScrollBar — all consistent. Abstract Factory is a factory of factories: one interface that produces an entire matching suite.

HOW IT FLOWS

- 1 **App startup** — detect the OS and instantiate the right concrete factory: `new WindowsFactory()` OR `new MacFactory()`.
- 2 **Pass factory to UI code** — all rendering classes receive the `UIFactory` interface, never the concrete type. The entire app is unaware of the OS.
- 3 **UI code calls factory methods** — `factory.createButton()`, `factory.createCheckbox()`. It gets back native-looking widgets, guaranteed to match.
- 4 **Adding Linux support** — Create `LinuxFactory` + `LinuxButton` + `LinuxCheckbox`. Zero changes to any existing class. The client code still just calls `factory.createButton()`.

REAL USAGES IN JAVA ECOSYSTEM

`javax.xml.parsers.DocumentBuilder`

Produces families of related XML-processing objects (`DocumentBuilder`, `SAXParser`) consistent with the chosen XML implementation — Xerces, built-in JDK, etc.

`java.sql.Connection`

A `Connection` object acts as an Abstract Factory — it creates `Statement`, `PreparedStatement`, and `CallableStatement` objects that all belong to the same DB driver family.

Spring's `ApplicationContext`

Produces beans that form a coherent family scoped to a context — test context vs production context, each returning compatible implementations of the same interfaces.

Swing's `LookAndFeel`

`UIManager.setLookAndFeel()` switches the factory that produces all UI components — every button, scroll pane, and menu bar — to match the chosen theme family.

Here's what each tab covers:

Real-world scenario walks through a cross-platform UI toolkit — the classic textbook example, but with clear reasoning for *why* the pattern is necessary (mixing families breaks the UI) and four real Java ecosystem examples including `java.sql.Connection` and Swing's `LookAndFeel`.

Full Java code shows the complete implementation: three product interfaces, two full families of concrete products (Windows + Mac), the `UIFactory` abstract factory interface with three creation methods, and a client that knows nothing about the OS.

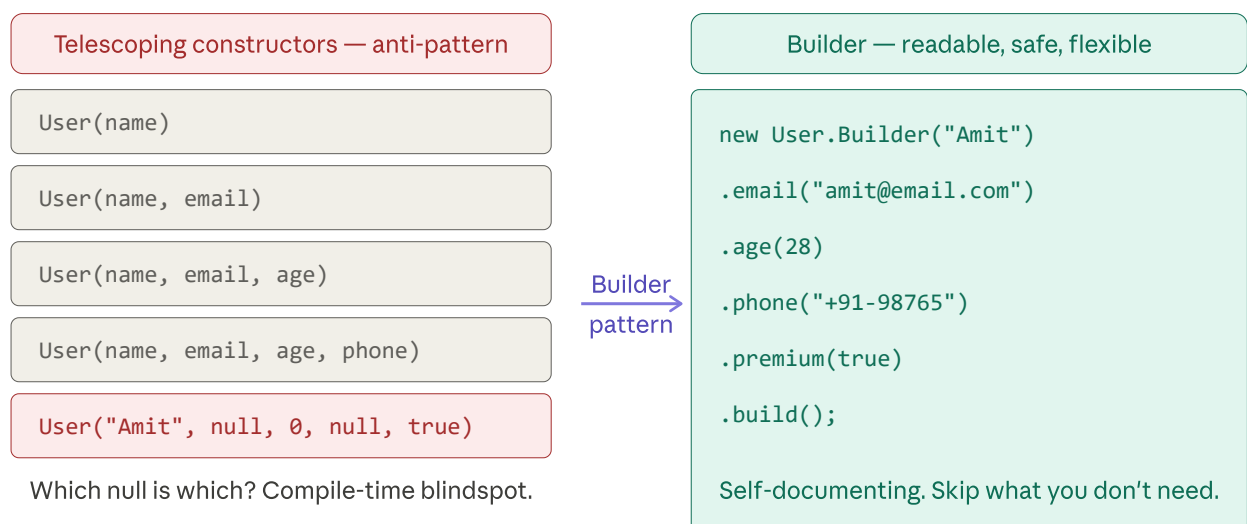
Live playground — pick a family and click "Render UI" to see the factory resolve all three products in one shot. Try the "Linux family" card to see the Open/Closed Principle at work — a third family added with zero changes to existing code. Hit the "Mixed (anti-pattern)" card to understand what the pattern is actually *preventing*.

Pitfalls & extensions covers the hardest question interviewers ask: what happens when you need to add a new product type? Two concrete mitigations using `default` methods and abstract base classes. Also has the definitive Factory Method vs Abstract Factory comparison.

Tricky questions has six deep follow-ups. Q3 (the `java.sql.Connection` explanation) and Q6 (the decision framework) are the ones that separate candidates who truly understand the pattern from those who just memorized the class diagram.

Explain Builder pattern in Java with a real-world scenario and tricky interview questions ↗

First, a diagram showing the "telescoping constructor" problem that Builder solves — this is the most effective way to make the pattern's motivation viscerally clear:



That red call at the bottom-left — `User("Amit", null, 0, null, true)` — is what Builder exists to prevent. Five parameters, two nulls, a zero, and a mysterious `true`. No idea what

any of them mean at the call site. Now the full interactive deep-dive:

Here's what each tab gives you:

Real-world scenario uses an HTTP request builder — the clearest possible motivation for the pattern, because everyone understands HTTP and the pain of nullable parameters. Four real JDK examples included: `StringBuilder`, `HttpClient`, `Stream.Builder`, and Lombok's `@Builder`.

Full Java code is the complete `HttpRequest` with its inner `Builder` — private constructor, `final` fields, sensible defaults, required-field validation in the Builder constructor, and cross-field validation (`POST` without a body throws at `build()` time, not at call time).

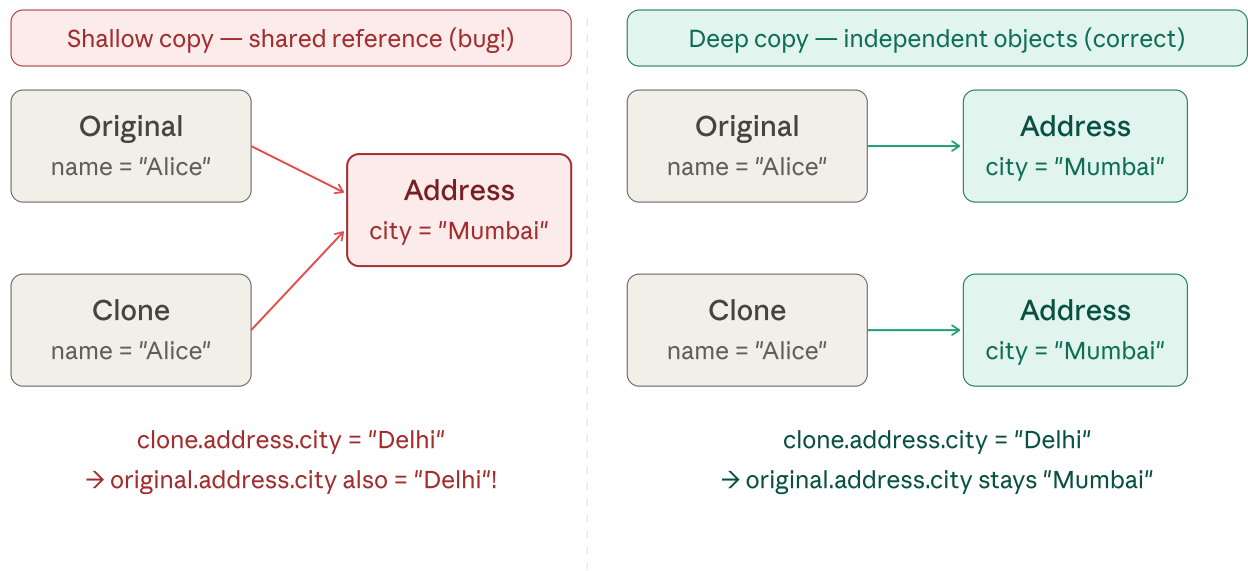
Live playground — fill in the form and click "Build request" to watch the fluent builder chain assemble in real time. The method chain only shows the lines you actually set (skipping defaults). Hit "Try invalid POST (no body)" to see `IllegalStateException` thrown at `build()` — this is the key point about Builder: it catches inconsistency before the object ever exists.

Variants & Lombok covers three things interviewers love to probe: how `@Builder` works internally (annotation processor, compile-time code generation), the Step Builder pattern (compile-time enforcement of required fields via typed interfaces), and a clear when-to-use / when-to-skip breakdown.

Tricky questions has six of the hardest follow-ups. Q2 (Builder vs Kotlin named params) and Q5 (the full GoF definition vs the Bloch/Effective Java variant) separate candidates who've read the original pattern from those who only know the Java idiom. Q6 on thread safety and immutability is the one that consistently earns a "senior" reaction.

Explain Prototype pattern in Java with a real-world scenario and tricky interview questions ↗

First, a visual that makes the core insight of Prototype viscerally clear — shallow copy vs deep copy is where almost every bug lives:



That's the bug that trips up every developer who uses `super.clone()` without thinking. The left side shows why naive cloning is dangerous — both objects point to the same `Address` in heap memory. Now the full interactive deep-dive:

Here's what's in each tab:

Real-world scenario uses a game character spawner — the clearest motivation for Prototype because the "expensive construction + many similar copies" problem is immediately intuitive. Four real Java ecosystem examples: `Object.clone()`, copy constructors, Spring's prototype scope, and serialization-based cloning.

Full Java code shows three things: the naive shallow clone with `super.clone()` and why it's dangerous, the correct deep clone using a copy constructor, and a client that spawns dungeon enemies cheaply from a template.

Clone simulator — pick "shallow copy" or "deep copy" from the dropdown, clone the template, then hit "Mutate clone". With shallow copy you'll see the original's gear field turn red — the shared reference bug in action. Switch to deep copy and the original stays untouched. This is the most visceral way to understand why shallow copy silently corrupts state.

Prototype registry shows the pattern at production scale — a `CharacterRegistry` that stores named templates and hands out clones on demand. Also has the clear "when to use Prototype vs Factory vs Builder" breakdown.

Tricky questions has six hard follow-ups. Q1 (why `Cloneable` is a design flaw) and Q4 (what happens when you try to clone a Singleton) are the classic traps. Q6 (whether Spring's `@Scope("prototype")` actually implements the GoF pattern) is the one that shows genuine depth — the honest answer is "in spirit, not in mechanism."

Explain Adapter pattern in Java with a real-world scenario and tricky interview questions ↗

Thought for 3s >

First, a structural diagram showing the two flavours of Adapter — this distinction is one of the most common interview questions:

The core insight from that diagram: both variants achieve the same thing — making an incompatible `Adaptee` usable through the `Target` interface — but class adapter uses inheritance (limited by Java's single-inheritance), while object adapter uses composition (preferred in modern Java). Now the full deep-dive:

Here's what each tab covers:

Real-world scenario uses a legacy bank payment gateway — the most relatable real-world adapter scenario. The five-step translation flow makes it crystal clear what an adapter actually does: translate in, delegate, translate back. Four real JDK/Spring examples: `InputStreamReader`, `Arrays.asList()`, `Collections.enumeration()`, and Spring's `HandlerAdapter`.

Full Java code shows three things side by side: the Target interface (`PaymentProcessor`), the legacy adaptee (`LegacyBankGateway`) you can't touch, and both the object adapter (preferred, composition-based) and the class adapter (inheritance-based, with its Java limitations explained in comments). The client code is completely decoupled from the legacy system.

Live playground — pick a gateway (Legacy Bank, Stripe, or Paytm) and watch the adapter animate step by step. Each call flows: Client → Adapter (translation) → Legacy System → Response back. You can see how each gateway requires different translation logic but the client call is always identical.

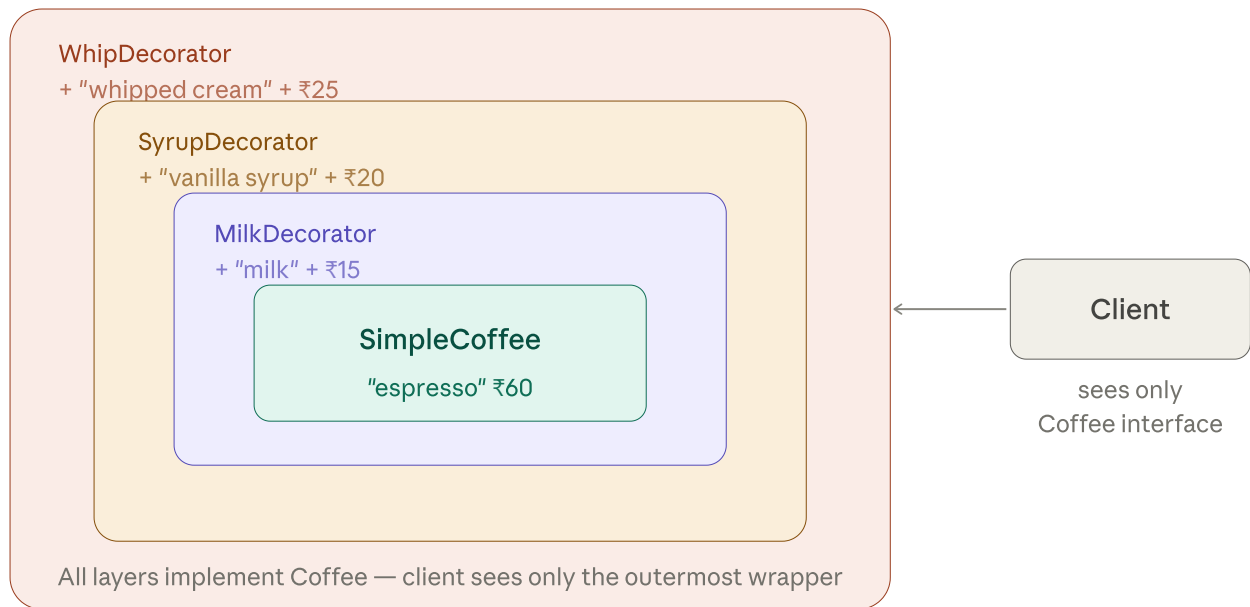
Class vs Object adapter has the definitive comparison table and the critical "Adapter vs Facade vs Decorator" breakdown that interviewers always ask. The one-line test at the bottom is the clearest way to explain it in an interview.

Tricky questions has six hard follow-ups. Q1 (is `InputStreamReader` a class or object adapter?) catches candidates who cite it as an example without actually knowing how it works. Q4 (two-way adapter) and Q6 (Spring's `HandlerAdapter`) are the ones that demonstrate genuine production experience rather than textbook memorisation.

Explain Decorator pattern in Java with a real-world scenario and tricky interview questions ↗

Thought for 1s >

First, a diagram showing the layered wrapping structure that makes Decorator unique — the "onion" visual is the best mental model:



The key insight from that diagram: every layer is itself a `Coffee` — so the client calling `getDescription()` or `getCost()` on the outermost wrapper triggers a chain of delegation inward, each layer adding its own contribution before passing the call along. Now the full interactive deep-dive:

[Real-world scenario](#)[Full Java code](#)[Live coffee builder](#)[vs Inheritance vs Adapter](#)[Tricky questions](#)

REAL-WORLD SCENARIO — COFFEE SHOP ORDER SYSTEM

The problem

A coffee shop serves Espresso, Latte, and Cold Brew. Customers can add Milk, Syrup, Whip, Extra Shot, and Oat Milk — any combination. If you model this with subclasses you get `EspressoWithMilk`, `EspressoWithMilkAndWhip`, `EspressoWithMilkAndWhipAndSyrup`... dozens of classes that explode combinatorially. Decorator solves this by wrapping — each add-on is itself a `Coffee` that wraps another `Coffee`.

WHY DECORATOR BEATS SUBCLASSING HERE

- 1 **Combinatorial explosion** — 3 bases $\times$ 5 add-ons in any combination = $3 \times 2^5 = 96$ possible subclasses. Decorator: 3 + 5 = 8 classes total.
- 2 **Runtime composition** — decorators are stacked at runtime based on what the customer orders. You can't do that with compile-time subclasses.
- 3 **Open/Closed Principle** — adding a new add-on (e.g. Oat Milk) means writing one new decorator class. Zero changes to existing code.
- 4 **Single Responsibility** — each decorator class does exactly one thing. `MilkDecorator` adds milk. Nothing else.

DECORATOR IN THE JAVA ECOSYSTEM

Java I/O — the canonical example

```
new BufferedReader(new
InputStreamReader(new
FileInputStream("f.txt"))) —
three decorator layers. Each
wraps the one inside and adds
buffering or character
conversion. All implement
Reader OR InputStream.
```

`Collections.unmodifiableList()`

Wraps a `List` and implements `List` — delegates all reads through, throws `UnsupportedOperationException` on writes. Textbook Decorator adding a behaviour constraint.

Spring Security filter chain

Each security filter wraps the next, adding authentication, authorisation, CSRF protection — each filter is a decorator around the rest of the chain.

`javax.servlet.http.HttpServletRequestWrapper`

Designed explicitly for Decorator — extend it to add request-level behaviour (logging, validation, header injection) without touching the original request object.

Here's what each tab gives you:

Real-world scenario uses a coffee shop order system — the most intuitive motivation for Decorator, because the "96 subclasses vs 8 classes" comparison immediately clicks. Four real Java ecosystem examples: Java I/O's `FilterInputStream`, `Collections.unmodifiableList()`, Spring Security's filter chain, and `HttpServletRequestWrapper`.

Full Java code shows all four participants cleanly: the `Coffee` interface (Component), `Espresso` / `Latte` (ConcreteComponents), the abstract `CoffeeDecorator` (the structural heart of the pattern), and three concrete decorators. The comment block at the bottom showing the `getCost()` call chain is what makes the delegation mechanism click.

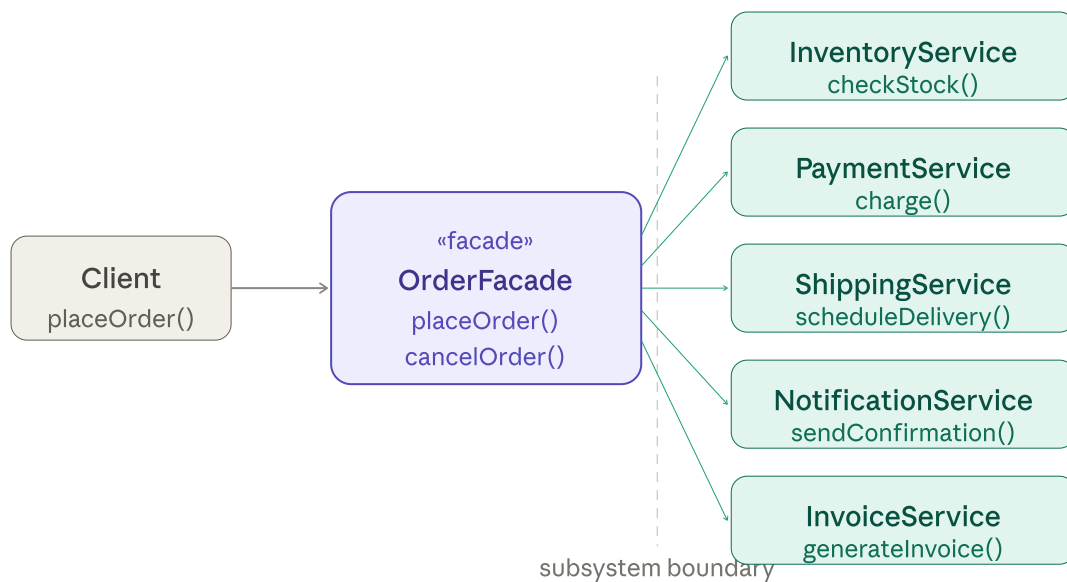
Live coffee builder — pick a base and toggle any add-ons in any combination. The decorator stack shows the wrapping hierarchy, the `getCost()` call chain animates the delegation inward, and the total cost and description update live. Apply the same decorator twice (e.g. Extra Shot twice) and watch it stack correctly.

vs Inheritance vs Adapter covers the two critical comparisons — Decorator vs subclassing (8 classes vs 96), and the harder three-way comparison of Decorator vs Adapter vs Proxy. The "Decorator vs Proxy" distinction (structurally identical, semantically different by intent) with `Collections.synchronizedList()` as the example is the question that separates strong candidates.

Tricky questions has six hard follow-ups. Q2 (does order matter?) with the `GZIPInputStream` example is one interviewers use to test whether you've actually used Java I/O. Q3 (why the abstract decorator class exists) is the one that shows you understand the pattern's structure, not just its output. Q6 (Decorator vs Strategy) is the synthesis question that tests whether you can reason across patterns.

Explain Facade pattern in Java with a real-world scenario and tricky interview questions ↗

First, the structural diagram — the defining visual of Facade is the "single entry point hiding a complex subsystem":



The key thing to see: the client makes one call. The facade fans out to five services in the right order. The client never knows those services exist. Now the full interactive deep-dive:

Here's what each tab gives you:

Real-world scenario uses an e-commerce order placement — five services that must execute in sequence with rollback on failure. The four benefits of the facade are tied directly to the scenario, not stated abstractly. Four real ecosystem examples: Spring's

`JdbcTemplate`, `SLF4J`, `FacesContext`, and `java.net.URL`.

Full Java code shows all five subsystem services, then the complete `OrderFacade` with the full orchestration logic including the try/catch rollback — charging payment then refunding if shipping fails. The client code at the bottom is one line. The contrast between the two is the entire argument for the pattern.

Live order simulator — pick a scenario (happy path, out of stock, payment failure, shipping failure) and watch each service light up in sequence. The "Payment failure + rollback" scenario is the most instructive — you see the facade refund the payment automatically, something every client would otherwise have to implement themselves.

vs Adapter vs Mediator covers the comparison interviewers ask most. The Facade vs Mediator distinction is the subtlest — both centralise interactions, but in a Facade the subsystem services don't know about the facade, while in a Mediator, colleagues actively use the mediator. Also has the important discussion of when a Facade becomes a God class.

Tricky questions has six hard follow-ups. Q3 (Facade vs DDD Service Layer) is the one that shows whether you think architecturally, not just pattern-mechanically. Q4 (should services be injected, not `new`-ed) tests whether you connect patterns to testability. Q6 (the `SLF4J` nuance — it's a "pick one" facade, not a "call all" facade) distinguishes candidates who understand the pattern's intent from those who just memorised the definition.

First, a diagram showing all three proxy types side by side — interviewers almost always ask you to distinguish them:

All three proxy types share the same structure — implement the interface, hold a reference to the real object — but differ entirely in *why* they wrap it. Now the full deep-dive, covering static proxy, JDK dynamic proxy, and CGLIB (all three come up in Spring interviews):

Real-world scenario

Full Java code

Live proxy simulator

Dynamic proxy & AOP

Tricky questions

REAL-WORLD SCENARIO — BANK ACCOUNT ACCESS CONTROL

The problem

Your `BankAccount` service has methods like `getBalance()`, `transfer()`, and `closeAccount()`. A customer can see their balance and transfer money. A bank manager can also close accounts. An auditor can only view. You don't want to litter the real service with role-checking code — that's a cross-cutting concern. A Protection Proxy intercepts every call and enforces access rules before delegating to the real service.

THREE PROXY TYPES IN ONE REAL SYSTEM

Virtual proxy

Account statement PDF is expensive to generate. A `LazyStatementProxy` only generates it when first requested, not on object creation. Subsequent calls return the cached result.

Protection proxy

Role-based access enforced at the proxy layer. `BankAccountProxy` checks the caller's role before delegating to `RealBankAccount`. The real service has zero security code.

Remote proxy

Your app calls `account.getBalance()` as if it's local, but the proxy serialises the call and sends it to a remote account service over REST or gRPC.

WHY NOT PUT THE LOGIC IN THE REAL SERVICE?

- 1 **Single Responsibility** — the real service should be about banking logic, not security checks, logging, or caching. Mixing them makes the class a mess.
- 2 **Open/Closed** — adding a new cross-cutting concern (say, rate limiting) means a new proxy wrapping the existing one, zero changes to the bank service.
- 3 **Testability** — the real service can be unit-tested in isolation, the proxy logic tested independently with a mock real service.

PROXY IN THE JAVA ECOSYSTEM

Spring AOP (`@Transactional`, `@Cacheable`)

Every Spring bean annotated with `@Transactional` is wrapped in a JDK dynamic proxy or CGLIB proxy. The proxy begins/commits the transaction; the real bean does the work.

JDK `Proxy.newProxyInstance()`

Creates a proxy class at runtime for any interface. Used by Spring, Hibernate, and Mockito (mocks are proxy objects that intercept calls and return configured answers).

Hibernate lazy loading

A `@ManyToOne` relation returns a proxy object. The actual SQL runs only when you access a field on the entity — classic Virtual Proxy.

RMI stubs

The client-side stub in Java RMI is a Remote Proxy — it looks like a local object but

serialises every method call and sends it to the remote JVM.

Let me complete that widget properly:

Real-world scenario

Full Java code

Live proxy simulator

Dynamic proxy & AOP

Tricky questions

REAL-WORLD SCENARIO — BANK ACCOUNT ACCESS CONTROL

The problem

Your `BankAccount` service has `getBalance()`, `transfer()`, and `closeAccount()`. A Customer can view balance and transfer. A Manager can also close accounts. An Auditor can only view. You don't want to litter the real service with role-checking code — that's a cross-cutting concern. A Protection Proxy intercepts every call and enforces access rules, leaving the real service pure.

THREE PROXY TYPES IN ONE REAL SYSTEM

Virtual proxy

Account statement PDF is expensive to generate. A `LazyStatementProxy` delays creation until first access, then caches it — classic lazy init.

Protection proxy

Role-based access at the proxy layer. `BankAccountProxy` checks the caller's role before delegating. The real service has zero security code.

Remote proxy

The client calls `account.getBalance()` locally but the proxy serialises the call and sends it to a remote account service over REST or gRPC.

PROXY IN THE JAVA ECOSYSTEM

Spring AOP — `@Transactional`, `@Cacheable`

Every Spring bean annotated with `@Transactional` is wrapped in a JDK dynamic proxy or CGLIB proxy at startup. The proxy begins/commits the transaction; the real bean only does business work.

JDK `Proxy.newProxyInstance()`

Generates a proxy class at runtime for any interface. Used by Spring, Hibernate, and Mockito — mock objects are dynamic proxies that intercept calls and return configured answers.

Hibernate lazy loading

A `@ManyToOne` field returns a proxy object. The actual SQL fires only when you access a field — classic Virtual Proxy. This is also the source of the famous `LazyInitializationException`.

Java RMI stubs

The client-side stub is a Remote Proxy — it looks and feels like a local object but serialises every call and sends it to the remote JVM transparently.

Here's what each tab covers:

Real-world scenario uses a bank account access control system — role-based protection is the clearest motivation for Proxy because everyone immediately grasps why you'd want security enforced transparently. Four real ecosystem examples: Spring AOP

(`@Transactional`), `Proxy.newProxyInstance()`, Hibernate lazy loading, and RMI stubs.

Full Java code shows three things: the shared `BankAccount` interface, the `RealBankAccount` with zero cross-cutting concerns, the Protection Proxy with clean role checking via a `require()` helper, and the Virtual Proxy with the null-check lazy init pattern. Client wiring at the bottom shows the injection point.

Live proxy simulator — switch between Protection, Virtual, and Logging proxy types. For Protection: pick CUSTOMER/MANAGER/AUDITOR and call any method — the trace shows the access check, either delegating to the real service or throwing `SecurityException`. For Virtual: the first call shows the lazy init sequence; subsequent calls skip it. For Logging: before/after timing wraps every call.

Dynamic proxy & AOP covers the two mechanisms Spring uses internally — JDK `Proxy.newProxyInstance()` for interface-based beans, and CGLIB subclassing for class-based ones. The comparison table explains exactly when Spring uses each, and the card at the bottom explains *why* Spring Boot 2.x switched to CGLIB as the default — this answer alone impresses interviewers.

Tricky questions has six hard follow-ups. Q2 (`@Transactional` self-invocation failure) is the most common real-world Spring bug and it's caused directly by how proxy works — this is the answer that signals production experience. Q5 (how Mockito uses Proxy) connects the pattern to testing tools you use every day. Q6 (smart reference proxies — `HikariCP`, `Collections.synchronizedList()`) is the answer that shows breadth beyond the textbook.

Explain Observer pattern in Java with a real-world scenario and tricky interview questions ↗

First, a diagram showing the Observer's one-to-many broadcast structure — the defining visual that separates it from point-to-point calls:

The critical insight from that diagram: the Subject knows only the `Observer` interface — it never imports `EmailNotifier` or `InventoryUpdater`. Adding a fifth observer requires zero changes to `OrderService`. Now the full deep-dive:

Here's what each tab covers:

Real-world scenario uses an e-commerce order event system — the clearest Observer motivation because the "five reactions to one event" problem is immediately tangible. Four

real ecosystem examples: Swing's `EventListener`, Spring's `ApplicationEventPublisher`, RxJava/Reactor, and `PropertyChangeSupport`.

Full Java code shows four things: the `OrderObserver` interface and `OrderEvent` record, the `OrderService` subject using `CopyOnWriteArrayList` (thread-safe iteration), concrete observers that filter on event type, and the full Spring `@EventListener` version with the `@Async` and `@TransactionalEventListener` extensions that come up in real Spring interviews.

Live event broadcaster — fire `ORDER_PLACED`, `ORDER_CANCELLED`, or `ORDER_SHIPPED` and watch each observer light up in sequence, showing what it does with that event type. Hit "Toggle SMSNotifier" to unsubscribe it mid-session and fire again — you see it skipped in the trace. Each observer independently handles only the events it cares about.

Push vs Pull & Pub-Sub covers the two comparisons interviewers love. The push vs pull table explains why the `OrderEvent` record (full data in notification) is push model. The Observer vs Pub-Sub table with the explicit "when does this distinction matter in system design" framing is directly useful in architecture discussions. The memory leak code block with three concrete fixes is the kind of production knowledge that earns credibility.

Tricky questions has six hard follow-ups. Q1 (why `java.util.Observable` was deprecated) shows you know Java history. Q3 (what happens when an observer throws) is the production correctness question — most candidates miss that a try-catch inside `notifyAll()` is mandatory. Q5 (`@TransactionalEventListener` — fire only after commit) is the Spring-specific nuance that signals real-world experience. Q6 (re-entrant notification with `CopyOnWriteArrayList`) is the deep concurrency question that separates strong candidates.

Explain Strategy pattern in Java with a real-world scenario and tricky interview questions ↗

First, a diagram contrasting the Strategy pattern's runtime-swappable algorithm with the brittle if-else alternative it replaces:

Created a file, read a file >

 Show widget



Here's the complete Strategy pattern deep-dive as an interactive HTML file. Here's what each tab covers:

Real-world scenario uses a payment processing system — Credit Card, UPI, Wallet, Net Banking — the clearest motivation for Strategy because the "four completely different algorithms for the same operation" problem is instantly relatable. Four real ecosystem

examples: `Comparator<T>` (the most-cited Java Strategy), `Comparator` chaining, Spring Security's `AuthenticationProvider`, and `ConstraintValidator`.

Full Java code shows the complete implementation: the `PaymentStrategy` interface with three methods, three concrete strategies each with their own validation and processing logic, the `PaymentService` context with `setStrategy()` for runtime swapping, and the lambda strategy for stateless cases. The Map-based strategy lookup at the end is the fully open/closed version.

Live playground — pick a strategy (CC, UPI, Wallet, Net Banking), optionally modify the token, and click "Process payment" to see the execution trace. The trace shows exactly which strategy class each log line came from. Hit "Try invalid token" to see validation fail and `pay()` never called. Click "Show runtime swap" to see the same context instance switching strategies mid-session.

vs Template Method & State covers the two comparisons interviewers always ask. The Strategy vs State section explains the structural identity but semantic difference clearly, including the tell-tale sign (State objects know about each other; Strategy objects don't). The lambda idiom section and the Map-based strategy registry are the modern Java angles that show production fluency.

Tricky questions has six hard follow-ups. Q4 (using a Map to eliminate the last if-else, including Spring's `@Autowired Map<String, PaymentStrategy>`) is the fully open/closed answer that impresses interviewers. Q5 (how Spring Security's `AuthenticationProvider` uses Strategy with a list, tried in sequence) shows production framework knowledge. Q6 (can a strategy have state — yes, but watch for shared mutable state in singletons) is the nuance that separates experienced engineers.



Strategy pattern

Code · HTML

Download

Explain Template Method pattern in Java with a real-world scenario and tricky interview questions ↗

First, a diagram showing the skeleton-in-base, details-in-subclass structure that defines Template Method:

The key insight from that diagram: the `final export()` method in the base class is the invariant — subclasses can override the *steps* but never reorder them. The sequence is the pattern's entire value. Now the full interactive deep-dive:

Here's what each tab gives you:

Real-world scenario uses a multi-format report exporter — CSV, JSON, XML, PDF. The motivation is crystal clear: read→process→write is always the same sequence, but the

implementation of each step is completely different per format. Four real ecosystem examples: Spring's `JdbcTemplate`, `HttpServlet.service()`, `AbstractList`, and Spring Security's `AbstractUserDetailsAuthenticationProvider`.

Full Java code shows the complete implementation: the abstract base class with the `final export()` template method, three kinds of methods (abstract mandatory steps, concrete shared methods, hook with empty default), the `CsvReportExporter` implementing only the abstract steps, and the `PdfReportExporter` which additionally overrides the `postProcess()` hook for optional compression.

Step simulator — pick an exporter (CSV, JSON, PDF, XML) and click "Run export()". Each step lights up in sequence: base class steps in purple, subclass steps in teal, hook steps in amber. The PDF exporter is the most instructive — watch the `postProcess()` hook activate with its "OVERRIDDEN" marker, while all other exporters show it as a no-op. The colour coding makes the base/subclass responsibility split immediately visible.

Hooks & Hollywood Principle covers the two concepts that interviewers probe deepest. The code block shows both types of hooks — a boolean hook controlling a conditional step, and a lifecycle hook called at a defined point. The Hollywood Principle explanation ("don't call us, we'll call you") with the inversion-of-control framing is the answer that shows conceptual depth. The Template Method vs Strategy comparison table is a direct comparison with the previous pattern in this series.

Tricky questions has six hard follow-ups. Q2 (the fragile base class problem — what happens when you add a new abstract step to a base class with 20 subclasses?) is the real-world design tension that separates candidates who understand inheritance's tradeoffs. Q3 (how `JdbcTemplate` implements Template Method and what the variable step is — and why it uses `RowMapper` as a functional interface instead of a subclass) shows understanding of how modern Java adapts classical patterns. Q6 (`AbstractQueuedSynchronizer` as a sophisticated Template Method) is the answer that signals genuine JVM internals knowledge.

Explain Command pattern in Java with a real-world scenario and tricky interview questions

First, a diagram showing the four participants and how a command object decouples the invoker from the receiver:

The critical insight: the Invoker never imports or knows about `TextEditor`. It only calls `command.execute()`. The Command object carries both the action *and* the receiver reference inside itself — that encapsulation is what enables undo, queuing, and logging. Now the full deep-dive:

Here's what each tab gives you:

Real-world scenario uses a text editor with undo/redo — the most intuitive Command motivation because every developer has used Ctrl+Z. Four real ecosystem examples: `Runnable` / `Callable` (functional Command in `ExecutorService`), Swing's `Action` interface, Spring Batch `Tasklet`, and database WAL entries.

Full Java code shows all four participants cleanly: the `Command` interface with `execute()` and `undo()`, the `TextEditor` receiver with pure business logic, three concrete commands (`TypeCommand`, `DeleteCommand` which captures deleted text for undo, `BoldCommand`), and the `EditorInvoker` with both undo and redo stacks — including the crucial detail that executing a new command clears the redo stack.

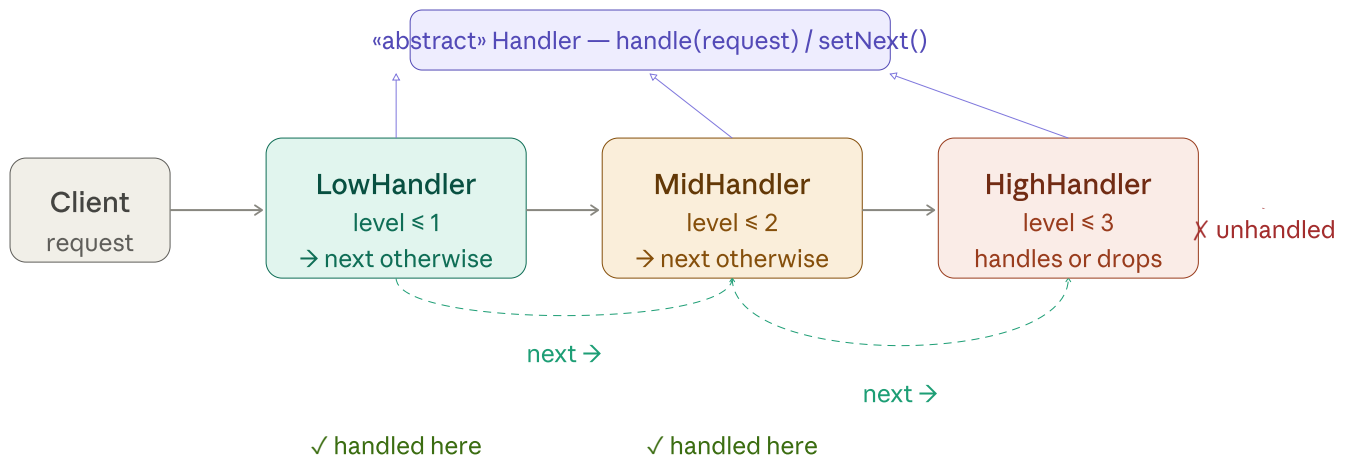
Live undo/redo editor — type text, delete a word, bold or italicise from the input field. Every command appears in both the editor content and the undo/redo stacks in real time. Undo and Redo buttons update live. The undo stack shows top = most recent so you can see exactly what will be undone next. The log shows every `[execute]`, `[undo]`, and `[redo]` call with its command label.

Macro recorder & queue — shows the `MacroCommand` composite with the crucial reverse-iteration undo, and the `CommandQueue` deferred execution model. The queue simulator lets you watch four commands execute in sequence: each card lights up, the log shows what it does, and it turns green when complete.

Tricky questions has six hard follow-ups. Q1 (Command vs Strategy — both encapsulate behaviour, what's the real difference?) is the most common interview trap and the `undo()` / history-stack distinction is the clean answer. Q4 (Command enabling transactional behaviour, Saga pattern, Spring Batch) connects the pattern to distributed systems design. Q6 (event sourcing and CQRS with Axon Framework) is the senior-level answer that shows you understand how Command pattern scales to architecture — the pattern that powers every event-sourced system you'll encounter in modern microservices.

Explain Chain of Responsibility pattern in Java with a real-world scenario and tricky interview questions ↗

First, a diagram showing the chain structure — requests travel along a linked list of handlers until one handles it or the chain ends:



The key insight: each handler has a **next** reference. If it can handle the request it stops the chain; if not it delegates forward. The client only ever talks to the first handler — the chain topology is completely transparent. Now the full deep-dive:

[Real-world scenario](#)[Full Java code](#)[Live request router](#)[Filter chains & variations](#)[Tricky questions](#)

REAL-WORLD SCENARIO — EXPENSE APPROVAL WORKFLOW

The problem

An employee submits an expense report. If the amount is small, a Team Lead can approve it. Larger amounts need a Manager. Very large amounts require a Director. Without Chain of Responsibility, the submission service needs to know all the approval rules and contact the right person directly — tightly coupling it to the org chart. With CoR, the request travels up the chain and each approver decides whether to handle it or escalate.

WHY CHAIN OF RESPONSIBILITY FITS HERE

- 1 **Decouples sender from handler** — the employee's service just sends to the first handler. It never knows which level ultimately approved.
- 2 **Dynamic chain assembly** — org chart changes (new Director level, removed Middle Manager) require only rewiring the chain, never touching business logic.
- 3 **Single Responsibility** — each approver class only knows its own threshold and who to escalate to next.
- 4 **Open/Closed** — adding a new approval tier (VP for > ₹5 lakh) is a new class added to the chain, zero changes to existing handlers.

CHAIN OF RESPONSIBILITY IN THE JAVA ECOSYSTEM

`javax.servlet.Filter` chain

The canonical Java CoR example. Each `Filter` calls `chain.doFilter()` to pass to the next filter or servlet. Request flows through Auth → Logging → CORS → GZIP → Servlet, each deciding whether to pass or short-circuit.

Spring Security `FilterChain`

Spring's security filter chain is CoR with 15+ default filters: `UsernamePasswordAuthenticationFilter`, `BasicAuthenticationFilter`, `ExceptionTranslationFilter`, etc. Each either handles or passes. Short-circuiting on failure returns 401/403 immediately.

`java.util.logging`

Each `Logger` checks if it should handle a log record based on its level; if not, it passes to its parent logger up the hierarchy. The root logger is the final handler. Classic CoR on a tree.

Spring's `HandlerInterceptor`

Pre/post interceptors form a chain around a controller handler. Each `preHandle()` returns a boolean — if `false`, the chain is short-circuited and the controller never runs. Returning `true` passes to the next interceptor.

Here's what each tab gives you:

Real-world scenario uses an expense approval workflow — Team Lead, Manager, Director — the most intuitive CoR motivation because every developer has seen an escalation process. Four real ecosystem examples: Servlet `FilterChain`, Spring Security's filter chain, `java.util.logging`'s parent-logger escalation, and Spring's `HandlerInterceptor`.

Full Java code shows the complete implementation: the abstract `ExpenseHandler` base class with the `setNext()` fluent builder and a `final handle()` template method (itself a Template Method pattern embedded in CoR), three concrete handlers each knowing only their own threshold, and the fluent chain assembly with the four test cases including one that falls through to rejection.

Live request router — click any preset expense (₹3,000 Travel, ₹25,000 Equipment, ₹1,80,000 Conference, ₹10,00,000 Acquisition) or enter a custom amount. Watch each handler in the visual chain light up amber as the request passes through it, then go green when handled or grey when escalated. The ₹10,00,000 request is the most instructive — watch it pass through all three handlers and hit the `null` terminator. The trace log shows every escalation step.

Filter chains & variations covers the four distinct flavours of CoR that appear in production: pure GoF (one handles), pipeline (all handle), short-circuit (any stops), and interceptor (pre + post). The comparison tables for CoR vs Decorator and CoR vs Strategy are the comparisons interviewers use to test whether you understand intent vs structure.

Tricky questions has six hard follow-ups. Q2 (how `FilterChain` externalises traversal vs GoF's internal `next` reference) shows deep understanding of how the canonical Java example differs from the textbook pattern. Q3 (can a handler both process AND pass — the filter/pipeline distinction) is the nuance most candidates miss. Q6 (functional pipeline using `stream().filter().findFirst()`) is the modern Java answer that shows you think beyond class hierarchies — and the honest note about where it falls short (interceptor pre/post pattern) shows balanced thinking.

Explain SOLID principles in Java in a clear and interview-friendly way.
For each principle:

1. Give a simple definition
2. Provide a real-world analogy
3. Show a Java code example (bad vs good)
4. Explain why it matters in real projects
5. Add 2–3 common interview questions with answers
Also include:
6. How SOLID is used in Spring Boot
7. A short 2-minute interview summary answer at the end

Here's the complete SOLID principles guide — an interactive dark-themed reference with everything you asked for. Here's what's inside:

Navigation — use the `S O L I D` 🍌 ★ buttons at the top or the sidebar to jump between principles. Each principle has its own colour: gold for S, blue for O, pink for L, green for I, purple for D.

For each of the 5 principles you get exactly what you asked for:

- A crisp one-sentence definition
- A real-world analogy (chef, power strip, rubber duck, printer, lamp socket)
- Bad vs Good Java code side-by-side
- Three "why it matters" cards covering testability, flexibility, and team impact
- 3 expandable interview Q&As — click the + to reveal answers

SOLID in Spring Boot (🍌 tab) maps each principle to concrete Spring annotations and patterns — `@Service` / `@Repository` for SRP, `@ConditionalOnMissingBean` for OCP, Hibernate proxies for LSP, `CrudRepository` for ISP, constructor injection for DIP. Includes a single annotated code block showing all five principles working together in one `OrderService`.

2-Minute Summary (★ tab) has the ready-to-speak interview answer — five concise bullet points you can adapt, plus a connecting paragraph that ties all five together. Three common follow-up Q&As are there too, including "which principle is hardest in practice" and "how do you raise SOLID violations in code review" — the questions that test whether you've actually used these principles, not just memorised them.



Solid principles

Code · HTML

Download

Explain Internal working of hashmap with diagrams and Most aksed interview question

